

第51讲 练习题与答案

先独立完成，再看答案。

练习题

1. 长度为 100 的单向 RNN 中，Token 1 的信息影响 Token 100，最简时间路径大约需要跨多少个递推步骤？Self-Attention 单层允许的直接路径约是多少？
2. 为什么 LSTM 的 Forget Gate 可以帮助长期记忆，但并没有让所有时间步在训练时完全并行？
3. GPT 使用 Causal Mask。为什么这不意味着训练时只能先算 Token 1，再算 Token 2？
4. 如果 Q 、 K 的 Shape 都是 (T,d) ， QK^T 的 Shape 是什么？为什么这会产生 T^2 级别的 score 数量？
5. “Transformer 的训练比 RNN 更容易并行”是否意味着 GPT 的文本生成也可以把未来 100 个 Token 一次全部生成？为什么？
6. 为什么不能看到本机 Python 循环比矩阵乘法慢很多，就直接声称 Transformer 比 RNN 快几百倍？
7. 在 Agent System 中，为什么 Transformer 内部 Token 计算可并行，并不意味着“调用工具→等待结果→继续决策”也可完全并行？
8. 一个模型支持 1M context window，是否自动说明它比 128K context 模型更擅长利用 80 万 Token 以前的信息？还需要什么证据？

答案与解释

1. RNN 约为 99 个时间递推边；单层允许全局交互的 Self-Attention 可建立约 1 层直接路径。
2. LSTM 改善 state 内信息保留和梯度路径，但结构仍是 $state_t = f(x_t, state_{t-1})$ ，所以 $state_t$ 仍要等待 $state_{t-1}$ 。
3. 训练时整段目标序列已知，可以一次计算所有 Q/K/V 和 $T \times T$ scores，再用上三角 Mask 把未来位置设为 $-\infty$ ，因此各行可矩阵化并行。
4. (T, T) 。每个 T 个 Query 都和 T 个 Key 产生一个 pair score，因此共有 $T \times T$ 个分数。
5. 不能。自回归生成的下一个新 Token 本身未知，后续 Token 依赖前面实际采样结果，所以生成时间轴仍串行。
6. Python 循环和高效矩阵 kernel 的实现层次不同；公平比较还需控制参数量、硬件、优化库、总计算、序列长度、batch、精度等。
7. 工具结果属于外部环境反馈，下一步决策必须等待真实结果，形成系统级数据依赖；模型内部并行不能删除外部因果依赖。
8. 不能。还需长程检索/推理 benchmark、位置分布实验、消融、有效利用率、不同距离的性能曲线等证据。

加分题：请用一句话分别解释“路径长度”“总 FLOPs”“wall-clock latency”“吞吐量”四个概念，说明为什么它们不能混为一个“效率”。